One Prompt, One Call: Systematic Prompt Design for Frozen-LLM Root Cause Localization in Microservices

Yifang Tian, Yiran Li*, Ruiwu Liu, Yifeng He, and Hans-Arno Jacobsen
Department of Electrical and Computer Engineering
University of Toronto, Canada
yifang.tian@mail.utoronto.ca, one.li@utoronto.ca, rev.liu@mail.utoronto.ca,
yifeng.he@mail.utoronto.ca, jacobsen@eecg.toronto.edu

***Abstract*—Root cause localization (RCL), which pinpoints the faulty service during a microservice incident, is a central AIOps task that must be accurate, fast, and inexpensive enough to run routinely at scale. Recent LLM-based RCL methods often rely on multi-turn agentic workflows that drive a model through tool use and intermediate analysis; these workflows can be effective, but require many calls and large token budgets per incident. Fine-tuned methods instead require labeled incidents and model-training overhead. We ask how far a single inference with a frozen language model can go when the prompt itself is systematically designed. We present SIRCL, a prompt-design framework for single-inference RCL that compiles heterogeneous telemetry into one prompt for one frozen-model call. SIRCL casts prompt construction as a search over five decisions governing both telemetry evidence and reasoning instructions, and fixes each decision through a sequential selection protocol with paired significance tests. On four held-out test sets, Qwen3.5-9B, an open-weight 9 B model, reaches 0.596 MRR with the selected SIRCL design, exceeding the strongest external baseline by +0.129 while consuming far fewer tokens than agentic pipelines. The fixed design transfers across seven LLM backbones, and a per-decision analysis shows what evidence the prompt should contain and how that evidence should be shaped.**

***Index Terms*—root cause localization, microservices, AIOps, observability, large language models, prompt design**

I. Introduction

Microservice architectures run much of the online economy [1], [2]: software systems are decomposed into hundreds of independently developed, deployed, and scaled services that communicate through network calls [1]. This design improves modularity and scalability, but complicates failure diagnosis. A fault in one service can propagate along service dependencies and surface as symptoms in components that are only indirectly affected: a faulty downstream service can inflate latency and errors in upstream callers, while a faulty upstream service can impose abnormal workload on downstream callees. The service that first alerts or shows a user-visible symptom is therefore often not the one that caused the incident [3], [4].

Root cause localization (RCL), which identifies the responsible service during an incident, is a central problem in modern AIOps platforms [4], [5]. Failed localization is expensive, as outages can cost thousands of dollars per minute [1]. Diagnosis typically draws on three runtime signals over a service dependency graph: metric time series, distributed traces, and logs [5], [6]. As microservice systems grow in scale and operational complexity, an RCL method must be accurate, fast enough to support timely mitigation, and cheap enough to run routinely across incidents.

Existing RCL methods include both non-large-language-model (non-LLM) and LLM-based approaches. Non-LLM methods use metric anomaly scoring, causal analysis, dependency-graph analysis, trace/log correlation, or learning-based telemetry fusion [3], [6], [7]. These methods are often efficient and interpretable, but have limited ability to use semantic cues in logs, task instructions, and cross-modality reasoning. LLM-based methods can use such information, but usually introduce additional cost. As illustrated in Figure 1, agentic and multi-agent systems perform multiple rounds of model inference, tool use, or intermediate analysis [8]–[12], requiring many model calls and large token budgets per incident. Fine-tuned LLM methods rely on large numbers of labeled incidents or reasoning trajectories and incur additional model-training overhead [13].

A single inference with a frozen LLM offers a simpler alternative: construct one diagnostic prompt containing the available telemetry evidence, submit it once, and parse the returned ranking. Here, frozen means that the model parameters are kept fixed; no weights are updated for a target system, benchmark, or incident. This setting avoids both multi-step inference overhead and per-system model training. The central challenge then becomes prompt construction: what telemetry evidence should be extracted, how should it be organized, and how should the model be instructed to reason over it? Existing one-shot RCL prompts are mostly hand-built [14]–[16]. Prior work on RCL prompt optimization [17] optimizes instructions and retrieves examples from historical incidents, but does not systematically study how telemetry evidence should be represented and arranged. More broadly, prompt optimization and context engineering have been effective for LLMs [18], [19], but their joint role in multi-modal microservice RCL remains underexplored.

* Corresponding author.

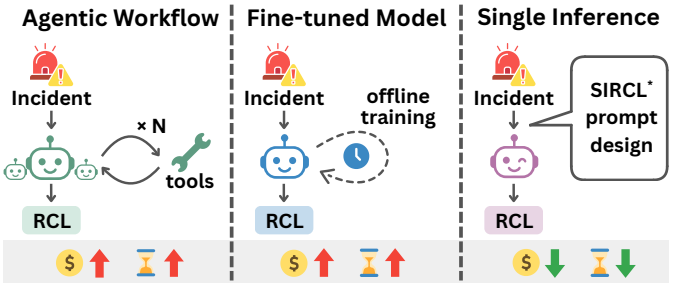


Fig. 1: Comparison of LLM-based RCL workflows. Agentic methods require iterative tool use and fine-tuned models require offline training, while SIRCL performs RCL with a single inference using a fixed prompt design.

We present SIRCL (Single-Inference Root Cause Localization), a systematic prompt-design framework for frozen-model RCL. Given an incident, SIRCL derives system topology, applies telemetry analyzers to summarize metrics, traces, and logs into compact evidence artifacts, arranges the selected artifacts into one prompt, adds optional guidance and reasoning instructions, and queries the model once. At inference time, SIRCL requires no fine-tuning, tool-calling loop, or follow-up LLM turn.

The prompt-construction space is combinatorial: evidence analyzers, modality sequencing, prompt layout, auxiliary guidance, and reasoning instructions interact to determine what the model sees and how it reasons. SIRCL makes this space searchable through a sequential selection protocol. Each design variable is evaluated on development incidents, and a candidate is adopted only when it improves over the incumbent on disjoint validation incidents. This procedure keeps the search feasible, reduces overfitting, avoids per-test-benchmark prompt tuning, and produces a *fixed* design that can be applied unchanged to new benchmarks and model backbones.

On four held-out test sets, the selected design reaches an average MRR of 0.596 with one call to Qwen3.5-9B, a 9B open model. It exceeds the strongest external baseline by $+0.129$ MRR and the strongest LLM-based baseline by $+0.144$ MRR, while using far fewer tokens than multi-turn LLM workflows. The same fixed design transfers across seven LLM backbones; stronger proprietary models can further improve accuracy, with Claude-Opus-4.7 reaching 0.654 average MRR. These results show that a carefully selected single-inference prompt can recover much of the benefit often sought through more expensive agentic or fine-tuned workflows.

Our contributions are:

1) We formulate microservice RCL as a single frozen-model inference and define a prompt-design space covering telemetry evidence, prompt organization, auxiliary guidance, and reasoning instructions.
2) We introduce a sequential design-selection protocol that makes prompt construction systematic, validation-driven, and reproducible.
3) We evaluate SIRCL on four held-out test sets, showing that the selected one-call design is accurate, efficient, transferable, and supported by per-decision analysis.

Algorithm 1: SIRCL inference pipeline

Input: Incident telemetry $I = (\mathcal{M}, \mathcal{T}, \mathcal{L})$; frozen language model $\mathcal{F}$; design variables Θ
Output: Ranked candidate root causes R

1 Derive system topology $(\mathcal{G}_D, \mathcal{H})$ from traces and deployment metadata, when available
2 Extract modality artifacts: $E_{\text{MET}} \leftarrow a_{\text{MET}}(\mathcal{M})$, $E_{\text{TRC}} \leftarrow a_{\text{TRC}}(\mathcal{T})$, $E_{\text{LOG}} \leftarrow a_{\text{LOG}}(\mathcal{L})$
3 $S \leftarrow \text{Seq}(E_{\text{MET}}, E_{\text{TRC}}, E_{\text{LOG}})$
4 $B \leftarrow \text{Layout}(S, \mathcal{G}_D, \mathcal{H})$
5 $C \leftarrow \text{Concat}(B, C_{\text{G}}, C_{\text{R}})$
6 $Y \leftarrow \mathcal{F}(C)$ `// Single LLM inference`
7 $R \leftarrow \text{Parse}(Y)$
8 **return** R

II. The Inference Pipeline and Design Variables

We formalize *single-inference root cause localization* (SIRCL) as a prompt-construction framework for microservice RCL with frozen language models. In SIRCL, an RCL instance is solved by constructing one diagnostic prompt that contains all information made available to the model: telemetry-derived evidence, structural information, task instructions, auxiliary guidance, and reasoning instructions. The model is then queried once, and its response is parsed into a ranked list of candidate root causes. Thus, SIRCL is defined not by the act of invoking a language model, but by the design choices that determine what evidence is extracted, which evidence is included, how it is laid out, and how the model is instructed to reason over it.

Given an incident $I = (\mathcal{M}, \mathcal{T}, \mathcal{L})$ over an analysis window, the raw inputs are metric time series $\mathcal{M}$, distributed traces $\mathcal{T}$, and event logs $\mathcal{L}$. SIRCL also uses derived structural information, including the service dependency graph $\mathcal{G}_D$ and, when available, the deployment hierarchy $\mathcal{H}$ that maps services to pods and pods to host nodes. For a frozen language model $\mathcal{F}$, an instantiation of SIRCL constructs a prompt C, obtains a single model output $Y = \mathcal{F}(C)$, and returns a ranking R over candidate components, such as services, pods, or nodes depending on the benchmark granularity.

Algorithm 1 gives the end-to-end inference pipeline. Line 1 derives structural context from the incident, including service dependencies and deployment hierarchy when available. Line 2 applies one analyzer per telemetry modality, producing compact metric, trace, and log artifacts. Lines 3–4 choose an ordered non-empty sequence of modality artifacts and lay them out with structural information into a prompt body B. Line 5 assembles the final prompt by concatenating B, optional auxiliary guidance C_{G}, and optional reasoning instruction C_{R}. Lines 6–7 query the frozen model once and parse its output into the root-cause ranking. No intermediate tool calls, follow-up turns, or model fine-tuning are performed during inference.

TABLE I: Candidate space for telemetry analyzers.

Analyzer	Mechanism	Reference
Metric analyzers		
MET−Z	Per-metric moving-window z-score at the trace-derived anomaly time	[20]
MET−R	Robust per-metric z-score from the median and inter-quartile range	[7]
MET−S	Per-service severity: peak $\|z\|$ per metric, summed over the service	[21]
MET−C	Marginal χ^2 dependence of each metric on the fault nodes	[22]
Trace analyzers		
TRC−H	Per-operation latency-percentile and status histograms, fault vs. baseline	[21]
TRC−G	Call graph with edges weighted by call count and mean duration	[23]
TRC−L	Spans ranked by exclusive-latency and call-count log-fold change vs. baseline	[24]
TRC−A	Per-operation support, confidence, and Jaccard association with failed traces	[25]
Log analyzers		
LOG−T	Per-service Drain-template frequencies over the fault window	[21]
LOG−R	Per-service error-keyword rate against a pre-incident baseline	[3]
LOG−F	Per-service error-line counts with top-K frequent tokens	[8], [26]
LOG−B	Content filter that drops low-information Drain templates	[27]

The prompt construction is governed by the design configuration $\Theta = (a_{\text{MET}}, a_{\text{TRC}}, a_{\text{LOG}}, \text{Seq}, \text{Layout}, C_{\text{G}}, C_{\text{R}})$. For each variable, we define a finite candidate set. The analyzer, layout, guidance, and reasoning instruction candidates are curated to cover representative mechanisms from prior RCL systems, LLM prompting strategies, and diagnostic controls. The modality-sequencing candidates enumerate all ordered non-empty sequences of metric, trace, and log artifacts. The design space is the Cartesian product of these candidate sets. The remainder of this section defines these candidate sets.

Telemetry Analysis. We pick one analyzer per modality, each mapping raw telemetry to a compact artifact (E_{MET}, E_{TRC}, E_{LOG}) that exposes anomalous components for the later selection, ordering, and formatting choices. It therefore determines which metric, trace, and log signals reach the frozen model before any prompt-construction choice is applied.

We define three modality-specific candidate sets, with four analyzers for each modality (Table I); each name combines a modality prefix with a mechanism suffix, for example, MET−Z, TRC−L, and LOG−R. Within each modality, the candidates cover different levels of evidence abstraction, ranging from less-processed distributions and structures (latency or template histograms, the weighted call graph) to fully reduced per-candidate anomaly scores (z-scores, exclusive-latency ranks, error-rate deltas). This design allows us to study not only which telemetry modality is included, but also how much preprocessing and aggregation should be applied before the evidence is given to the model. Each analyzer follows a published mechanism and is normalized to the same downstream interface: it maps raw telemetry from one modality to a structured evidence artifact used in prompt construction.

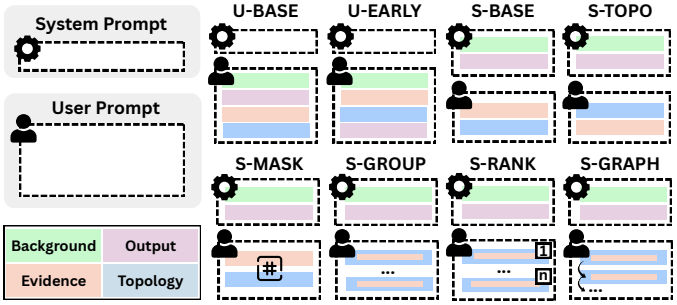


Fig. 2: Candidate space for prompt layout (Layout).

Modality Sequencing. Here, we determine the ordered modality sequence placed in the evidence block. Given the modality artifacts produced by the selected analyzers, we define a candidate sequence S as a non-empty ordered sequence over $\{\mathcal{M}, \mathcal{T}, \mathcal{L}\}$. Thus, S jointly specifies which modality artifacts enter the prompt and their relative order. We enumerate all 15 possible sequences (3 single-, 6 two-, and 6 three-modality).

Prompt Layout. Layout controls how the fixed evidence and structural context are arranged within the single LLM inference, including whether background and output instructions are placed in the system prompt or together with the incident context in the user prompt. We compare eight layouts, shown in Figure 2. The two U−⋆ layouts place all content in the user prompt. U−BASE is a vanilla layout where background and output instructions appear first, followed by the selected evidence and system topology $(\mathcal{G}_D, \mathcal{H})$. U−EARLY is the same except that it moves the evidence before the output instruction. The S−⋆ layouts place background and output instructions in the system prompt and incident-specific context in the user prompt. S−BASE keeps evidence before topology, while S−TOPO exchanges their positions. S−GROUP groups evidence by service, S−RANK orders service blocks by a fused ranking from telemetry analyzers, and S−GRAPH arranges service evidence along the dependency graph. Finally, S−MASK anonymizes service, pod, and node identifiers, probing reliance on human-readable names.

Auxiliary Guidance. This variable C_{G} is an optional piece of guidance text, inserted immediately after the background. It provides case-agnostic diagnostic heuristics for RCL, supplementing the telemetry evidence with general guidance on how to interpret failure symptoms. The default NONE leaves C_{G} empty. HEUR provides a compact set of literature-derived heuristics: Claude-Opus-4.7 distills general RCL rules from a curated set of recent RCL papers [9], [11], [20], [28]. This variant tests whether adding diagnostic domain knowledge improves accuracy when the telemetry evidence is unchanged.

We also evaluate mined guidance from labeled RCL predictions, but these variants are *not eligible* for selection because they use ground-truth outcomes. For each dataset, MINE uses DeepSeek-v4-pro to distill incorrect predictions from 20 held-out cases into at most ten one-sentence diagnostic rules, capturing recurring failures rather than case-specific

TABLE II: Candidate space for reasoning instruction C_R.

Instruction	Mechanism	Reference
NONE	Emits the ranking directly	-
THINK	Applies vanilla chain-of-thought instruction before answering	[18]
STEPS	Walks through the metrics and logs, reasons about propagation through topology, reflects on origin-vs-symptom evaluation	[13]
CITE	Supports each ranked service with quoted telemetry, allowing no ungrounded guesses	[12]
VERIFY	Drafts a ranking, poses two evidence-answerable questions about the top candidate, answers them, then revises the ranking or justifies keeping it	[30]
WALK	Seeds root-cause hypotheses at the anomalous services, then walks at most two hops along the call graph for validation	[12]
TRIAGE	Classifies the fault into a coarse class, then follows a class-specific checklist of standard operating procedures	[9]
ROT	Reasons over the trace while gathering metric evidence to form and iteratively refine a root-cause hypothesis	[20]

answers. Three variants alter the mining process: MINE−ALL uses both correct and incorrect predictions, MINE−G55 uses GPT-5.5 [29] as the distiller, and MINE−2X uses incorrect predictions from 40 cases.

Reasoning Instruction. C_{R} replaces the default output instruction and specifies how the model should reason before returning the final ranking. Unlike C_{G}, which supplies diagnostic heuristics about telemetry interpretation, C_{R} controls the reasoning procedure requested from the model. It does not introduce additional model calls; any extra cost appears only as additional tokens in the same single response. We compare seven instruction variants against the default NONE, which requests the ranking directly (Table II).

III. Systematic Prompt Design

A. Search Space Size and Initial Design

The SIRCL pipeline in Algorithm 1 defines a family of single-inference RCL methods parameterized by a design configuration Θ. The candidate choices induce a large combinatorial design space. Within our candidate space, the modality analyzers and modality sequence are coupled: if a sequence contains only a subset of modalities, only the analyzers for those modalities affect the resulting prompt. Thus, the analyzer–sequence choices contribute $3\cdot 4+6\cdot 4^2+6\cdot 4^3 = 492$ effective configurations, corresponding to the single-, two-, and three-modality sequences. Combined with 8 evidence layouts, 2 auxiliary guidance configurations, and 8 reasoning instructions, this yields $492\times 8\times 2\times 8 = 62{,}976$ possible configurations. As evaluating one configuration over hundreds of incidents requires one LLM call each, exhaustive enumeration would require tens of millions of model calls and is infeasible in both runtime and monetary cost.

We therefore begin from a deliberately simple, non-optimized design configuration Θ_0, denoted by SIRCL−0. For the modality-specific analyzers, SIRCL−0 sets $a_{\texttt{MET}} =$ MET−S, $a_{\texttt{TRC}} =$ TRC−H, and $a_{\texttt{LOG}} =$ LOG−T, all adapted from

Algorithm 2: Sequential design selection for SIRCL

Input: Initial design Θ_0; ordered design variables $\Pi = (\theta_1, \ldots, \theta_K)$ with candidate sets $\mathcal{Q}(\theta_k)$; frozen LLMs $\mathbb{F}$; development and validation incidents $\mathcal{D}_{\mathrm{dev}}, \mathcal{D}_{\mathrm{val}}$; primary metric μ; significance level α

Output: Selected design $\Theta^\star$

1 $\Theta \leftarrow \Theta_0$
2 **for** $k \leftarrow 1$ **to** K **do**
3 **foreach** $q \in \mathcal{Q}(\theta_k)$ **do**
4 $\mathbf{s}_q^{\mathrm{dev}} \leftarrow \mathrm{Eval}(\Theta[\theta_k \leftarrow q], \mathbb{F}, \mathcal{D}_{\mathrm{dev}}, \mu)$
5 $q^\star \leftarrow \arg\max_{q\in\mathcal{Q}(\theta_k)} \overline{\mathbf{s}_q^{\mathrm{dev}}}$
6 **if** $q^\star \neq \Theta[\theta_k]$ **then** // validation
7 $\Theta' \leftarrow \Theta[\theta_k \leftarrow q^\star]$
8 $\mathbf{s}_{\Theta'}^{\mathrm{val}} \leftarrow \mathrm{Eval}(\Theta', \mathbb{F}, \mathcal{D}_{\mathrm{val}}, \mu)$
9 $\mathbf{s}_{\Theta}^{\mathrm{val}} \leftarrow \mathrm{Eval}(\Theta, \mathbb{F}, \mathcal{D}_{\mathrm{val}}, \mu)$
10 $p \leftarrow \mathrm{PairedTest}\big(\mathbf{s}_{\Theta'}^{\mathrm{val}}, \mathbf{s}_{\Theta}^{\mathrm{val}}\big)$
11 **if** $p < \alpha$ **then**
12 $\Theta \leftarrow \Theta'$
13 **return** Θ *as* $\Theta^\star$

TORAI [21], a state-of-the-art non-LLM RCL method. For modality sequencing, SIRCL−0 includes all three modalities in the intuitive order MET−TRC−LOG. It further sets Layout = U−BASE, $C_G =$ NONE, and $C_R =$ NONE. These choices define a coherent vanilla prompt rather than an empirically optimized starting point. The design-selection procedure below then promotes a more specialized option only when it provides statistically supported improvement over the current design; otherwise, the simpler incumbent is retained.

B. Sequential Design Selection

To make the search tractable, we determine the design variables in a single ordered pass. Let $\mathbf{a} = (a_{\texttt{MET}}, a_{\texttt{TRC}}, a_{\texttt{LOG}})$ denote the analyzer group. We define the ordered variable sequence as $\Pi = (\mathbf{a}, \mathrm{Seq}, \mathrm{Layout}, C_G, C_R)$. For each variable $\theta_k \in \Pi$, $\mathcal{Q}(\theta_k)$ denotes the finite candidate set defined in Section II. This sequence follows the order in which the corresponding choices are applied in Algorithm 1: raw telemetry is first summarized into modality artifacts, the artifacts are then arranged into an ordered modality sequence, the selected artifacts are organized into an evidence block, and guidance and reasoning instructions are finally added to the prompt.

The analyzer group is a single stage that resolves three modality-local choices. Each candidate analyzer is scored only on its own modality under a fixed prompt context, so the comparison focuses on the quality of the modality artifact before the later sequencing and layout variables are introduced. The three selected analyzers enter Θ jointly before the remaining variables are processed.

Selection Rule. Algorithm 2 separates candidate screening from statistical confirmation. Line 1 initializes the incumbent design to Θ_0. Lines 2–12 process the design variables in

the order specified by Π. For the current variable θ_k, each candidate $q \in \mathcal{Q}(\theta_k)$ is evaluated on the development incidents while all other variables remain fixed, producing a score vector $\mathbf{s}_q^{\mathrm{dev}}$ over incidents and backbones. Line 5 selects $q^\star$, the candidate with the highest selection score $\overline{\mathbf{s}_q^{\mathrm{dev}}}$, defined below. If $q^\star$ is already the incumbent value, the algorithm makes no changes and moves to the next variable. Otherwise, Lines 7–10 evaluate the proposed Θ' and the incumbent Θ on validation incidents, and run a paired significance test between their score vectors. Lines 11–12 accept the update only when the validation test favors Θ' at level α; otherwise, the incumbent is retained. Line 13 returns the final incumbent as $\Theta^\star$.

This update rule preserves prompt simplicity unless a more specialized choice is supported by validation evidence. The initial design uses straightforward defaults for sequencing, layout, auxiliary guidance, and reasoning: all modalities are included in a prespecified order, the evidence block uses the basic layout, and no extra guidance or reasoning instruction is added. A more elaborate alternative can be adopted only if it ranks highest during development screening and then significantly improves over the incumbent on validation. For example, the default $C_R = \texttt{NONE}$ is retained unless an explicit reasoning instruction provides a validated improvement.

The score vector returned by Eval contains paired per-incident scores under the primary metric μ for every LLM backbone in $\mathbb{F}$. The overline in Line 5 denotes the selection score: for each backbone, we average the metric across incidents within each development dataset, average the resulting dataset-level scores, and then average across backbones. This gives equal weight to each dataset and each backbone. The development–validation split, backbone LLMs, primary metric μ, and statistical test are specified in Section IV-A.

The sequential design-selection algorithm evaluates a number of configurations proportional to the sum of the candidate-set sizes rather than their product, allowing prompt design to complete within a practical time and cost budget. The algorithm makes a single pass over the ordered variables: once a variable is processed, it is not revisited. We therefore use it as an interpretable and reproducible selection strategy, without claiming that it finds the best configuration in the full Cartesian-product search space. The resulting configuration, denoted by $\texttt{SIRCL}^\star$ in the evaluation, is then locked and applied unchanged to held-out benchmarks, systems, and model backbones. Thus, $\texttt{SIRCL}^\star$ is not a per-benchmark prompt tuned after seeing test cases, but a *fixed* design selected from development and validation evidence, with its generalization capabilities subsequently evaluated through testing.

IV. Evaluation

We structure the empirical evaluation of the SIRCL framework on three research questions (RQs):

- **RQ1.** Is the design configuration selected by SIRCL effective and efficient for RCL in microservices?
- **RQ2.** How are the prompt design options selected for $\texttt{SIRCL}^\star$, and what is their contribution to performance?
- **RQ3.** Does the selected SIRCL design transfer across LLM backbones, and are the results reproducible?

TABLE III: Microservice incident datasets.

Dataset	Label	Ref.	Root-cause level	Dev.	Val.	Test
RCAEval-OB	R-OB	[6]	Service	54	36	–
AIOps-2022	A-22	[31]	Service/pod/node	60	40	140
AIOps-2025	A-25	[32]	Service/pod/node	60	40	240
AegisLab	Aegis	[3]	Service	–	–	100
RCAEval-TT	R-TT	[6]	Service	–	–	90

A. Experimental setup

Datasets. We draw incident subsets from five public benchmarks (Table III): RCAEval-OB (R-OB), AIOps-2022 (A-22), AIOps-2025 (A-25), AegisLab (Aegis), and RCAEval-TT (R-TT). Design selection uses only R-OB, A-22, and A-25. For each of ten random seeds, we construct a stratified 60/40 development–validation split, run the procedure in Algorithm 2, and report results averaged over the splits. The test pool is untouched during design selection and consists of held-out A-22 and A-25 incidents, together with Aegis and R-TT. For the mined-guidance analysis only, we reserve a further 60 A-22 and 60 A-25 incidents, disjoint from all of the above.

Metrics. Following standard RCL practice [4], [6], [13], we report mean reciprocal rank (MRR) as the primary effectiveness metric. For an incident, reciprocal rank is the inverse rank of the first correct root-cause entity in the returned list; MRR averages this quantity over incidents. We also report accuracy@1 (A@1) and accuracy@3 (A@3) as secondary top-k measures, denoting the fraction of incidents for which a correct entity appears within the first one or three returned candidates. Cost per incident is measured by wall-clock time in seconds, total tokens, the number of LLM calls, and API cost in USD using provider-listed June 2026 rates. For each candidate update in SIRCL (Line 10 in Algorithm 2), we compare the proposed design with the incumbent on the validation split using a one-sided Wilcoxon signed-rank test [33] and set $\alpha = 0.05$; the update is accepted only when $p < \alpha$.

Baselines. We compare $\texttt{SIRCL}^\star$ with eleven classical, non-LLM RCL baselines and four LLM-based RCL baselines on the held-out test sets. Among the non-LLM baselines, CIRCA [34], RCD [22], MicroCause [35], and BARO [7] are metric-based causal or statistical localization methods; HeMiRCA [36] is trace-driven and correlates trace anomalies with metrics; and SimpleRCA [3], Nezha [37], PDiagnose [26], TORAI [21], Eadro [38], and DiagFusion [39] combine telemetry signals (metrics, logs, and traces), ranging from alert-count heuristics to learned multimodal fusion.

The LLM-based baselines cover different agentic and multi-agent RCL workflows. OpenRCA [11] has an autonomous tool-augmented agent that writes and runs code to query telemetry and iteratively narrow down the suspect set. Flow-of-Action [9] follows encoded SRE standard operating procedures to classify, scan, and judge. LocaleXpert [28] renders each modality as natural language for expert agents that collabora-

TABLE IV: RCL performance on test sets. Best bold, second underlined; fine-tuned LLM results are shown for reference.

	Aegis			R-TT			A-22			A-25			Macro average			Per-case cost		
Method	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	Time (s)	Tokens	Calls
Non-LLM baselines																		
CIRCA	0.202	0.090	0.210	0.636	0.533	0.711	0.185	0.110	0.150	0.196	0.140	0.170	0.305	0.218	0.310	559.7	–	–
RCD	0.069	0.030	0.110	0.151	0.100	0.189	0.183	0.100	0.270	0.158	0.110	0.210	0.140	0.085	0.195	47.3	–	–
MicroCause	0.091	0.020	0.070	0.330	0.178	0.400	0.159	0.050	0.157	0.208	0.079	0.204	0.197	0.082	0.208	**0.4**	–	–
BARO	0.227	0.100	0.220	0.706	0.667	0.822	0.293	0.079	0.350	0.249	0.114	0.205	0.369	0.240	0.399	1.1	–	–
HeMiRCA	0.355	0.240	0.370	0.322	0.211	0.344	0.329	0.250	0.340	0.289	0.200	0.320	0.324	0.225	0.344	1.5	–	–
SimpleRCA	0.202	0.080	0.160	0.709	0.644	0.733	0.141	0.064	0.093	0.277	0.179	0.283	0.332	0.242	0.317	5.8	–	–
Nezha	0.298	0.180	0.370	0.681	0.556	0.833	0.151	0.093	0.179	0.239	0.125	0.350	0.342	0.239	0.433	258.0	–	–
PDiagnose	0.457	0.340	0.500	0.676	0.611	0.678	0.181	0.093	0.186	0.296	0.179	0.379	0.403	0.306	0.436	105.4	–	–
TORAI	0.390	0.230	0.490	0.785	0.689	0.789	0.284	0.143	0.343	0.257	0.113	0.267	0.429	0.294	0.472	2.0	–	–
Eadro	0.504	0.350	0.590	0.699	0.622	0.722	0.324	0.214	0.343	0.340	0.213	0.400	0.467	0.350	0.514	81.6	–	–
DiagFusion	0.487	0.330	0.530	0.437	0.289	0.511	0.270	0.171	0.271	0.218	0.083	0.258	0.353	0.218	0.393	265.9	–	–
LLM-based baselines																		
OpenRCA	0.081	0.081	0.081	0.045	0.045	0.045	0.057	0.057	0.057	0.046	0.046	0.046	0.057	0.057	0.057	160.8	515.9k	16.5
Flow-of-Action	0.460	0.400	0.540	0.733	0.733	0.733	**0.388**	**0.379**	0.400	0.227	0.221	0.233	0.452	0.433	0.477	180.4	115.8k	13.5
LocaleXpert	0.521	0.370	0.660	0.741	0.622	0.856	0.205	0.129	0.264	0.291	0.221	0.358	0.440	0.336	0.535	17.2	17.5k	6.0
mABC	0.171	0.130	0.210	0.196	0.122	0.233	0.125	0.093	0.157	0.149	0.129	0.158	0.160	0.119	0.190	174.1	103.2k	49.2
Fine-tuned LLMs (reference)																		
ThinkFL-8B	-	-	-	-	-	-	0.201	0.136	0.221	0.226	0.175	0.229	0.107	0.078	0.113	12.7	16.4k	6.8
FoundRoot-14B	0.383	0.290	0.420	0.516	0.467	0.544	0.540	0.392	0.636	0.487	0.333	0.579	0.482	0.371	0.545	78.8	18.3k	1.0
SIRCL-0	0.292	0.250	0.340	0.692	0.644	0.722	0.173	0.143	0.207	0.161	0.104	0.229	0.330	0.285	0.375	4.4	15.6k	**1.0**
SIRCL*	**0.635**	**0.560**	**0.730**	**0.993**	**0.989**	**1.000**	0.379	0.314	**0.450**	**0.378**	**0.317**	**0.458**	**0.596**	**0.545**	**0.660**	4.4	**12.8k**	**1.0**

"–" indicates that the method does not produce scorable root-cause components for the target benchmark.

tively produce an interpretable ranking. mABC [10] aggregates seven agents with a voting mechanism under bounded step limits. Additionally, we include two fine-tuned LLMs for microservice RCL, FoundRoot [13] (14B) and the agentic model ThinkFL [20] (8B), and report their results for reference.

LLM Configuration. Design decisions for SIRCL are selected using two frozen LLM backbones: DeepSeek-v4-pro [40], representing a larger state-of-the-art model, and Qwen3.5-9B [41], representing a smaller open model. The cross-method comparison against LLM-based baselines uses Qwen3.5-9B as a shared backbone for all methods, so differences reflect the RCL workflow rather than the underlying model. We choose Qwen3.5-9B because it is a strong and cost-effective open model that can be served locally for reproducible latency and token accounting. To assess backbone sensitivity, we further apply the locked SIRCL* configuration, without redesign or retuning, to five additional models: Kimi-K2.5 [42], GPT-5.4 [43], GPT-5.4-mini [44], Gemini-3.1-pro [45], and Claude-Opus-4.7 [46]. Qwen3.5-9B is served locally with vLLM [47] on a single NVIDIA H100, and all other models are accessed through vendor APIs. Unless an API disables the option, model temperature is set to 0; for APIs that expose reasoning-effort controls, we use the default setting.

B. RQ1: Overall Effectiveness and Efficiency

Table IV compares SIRCL* with classical and LLM-based baselines on four held-out test sets.

Overall RCL accuracy. SIRCL* achieves the best macro-average performance across all methods, with 0.596 MRR, 0.545 A@1, and 0.660 A@3. Its average MRR exceeds the strongest external baseline, Eadro, by $+0.129$, and the strongest LLM-based baseline, Flow-of-Action, by $+0.144$. The gains are consistent across most datasets: SIRCL* ranks first on Aegis (0.635 MRR), R-TT (0.993), and A-25 (0.378), and is a close second on A-22, where Flow-of-Action has slightly higher MRR (0.388 vs. 0.379) but SIRCL* achieves the best A@3 (0.450). FoundRoot and ThinkFL use supervised fine-tuning on labeled incidents from microservice systems that overlap with our test set, but still trail SIRCL* in average performance. These results show that SIRCL* provides the strongest overall accuracy across the held-out test sets.

Effectiveness of systematic prompt design. The comparison between SIRCL-0 and SIRCL* isolates the effect of the design-selection procedure. Starting from the vanilla configuration, SIRCL-0 obtains 0.330 average MRR, 0.285 A@1, and 0.375 A@3. After sequential design selection, SIRCL* improves these scores to 0.596, 0.545, and 0.660, respectively, yielding absolute gains of $+0.266$ MRR, $+0.260$ A@1, and $+0.285$ A@3. The gains appear on every test set, showing that the selected design improves the default prompt beyond one particular benchmark.

Cross-benchmark generalization. Aegis and R-TT are excluded from development and validation, so performance on these datasets measures transfer to unseen benchmarks. SIRCL* ranks first on both: on Aegis, it improves over the strongest external baseline, LocaleXpert, by $+0.114$ MRR; on R-TT, it reaches 0.993 MRR, improving over TORAI by $+0.208$. The R-TT result is nearly saturated, with 0.989 A@1 and 1.000 A@3, whereas the best prior scores on this benchmark remain substantially lower: 0.785 MRR from TORAI

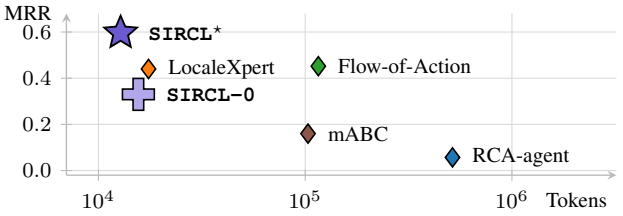


Fig. 3: Cost–accuracy tradeoff on test set. Upper left is better.

TABLE V: Telemetry-analysis candidate evaluation. ° marks SIRCL−0 default; bold marks the best development score.

	MRR by dataset			Qwen3.5-9B			DeepSeekv4-pro			Selection
Config	R-OB	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	Avg. MRR
Metric analyzers										
MET−Z	0.976	0.369	0.244	0.482	0.411	0.560	0.577	0.524	0.624	**0.530**
MET−R	0.948	0.257	0.157	0.409	0.343	0.469	0.499	0.433	0.553	0.454
MET−S°	0.887	0.207	0.174	0.425	0.383	0.481	0.420	0.390	0.451	0.423
MET−C	0.615	0.184	0.182	0.291	0.188	0.389	0.362	0.225	0.460	0.327
Trace analyzers										
TRC−H°	0.221	0.054	0.042	0.116	0.079	0.155	0.096	0.055	0.134	0.106
TRC−G	0.516	0.268	0.196	0.319	0.269	0.371	0.335	0.284	0.388	0.327
TRC−L	0.867	0.226	0.178	0.409	0.354	0.471	0.438	0.381	0.492	**0.424**
TRC−A	0.845	0.084	0.219	0.385	0.319	0.460	0.380	0.330	0.419	0.383
Log analyzers										
LOG−T°	0.304	0.078	0.197	0.183	0.136	0.256	0.203	0.155	0.275	0.193
LOG−R	0.251	0.179	0.240	0.189	0.129	0.260	0.258	0.195	0.316	**0.223**
LOG−F	0.154	0.168	0.243	0.165	0.145	0.187	0.211	0.153	0.247	0.188
LOG−B	0.215	0.166	0.212	0.141	0.127	0.156	0.254	0.185	0.304	0.197

Decision: validate the selected analyzers against the defaults; accept all three updates (a_{MET}: 0.405→0.500, p<0.001; a_{TRC}: 0.113→0.419, p<0.001; a_{LOG}: 0.191→0.224, p=0.007).

and 0.856 A@3 from LocaleXpert. Together, Aegis and R-TT show that the selected design transfers to benchmarks not used during prompt selection.

Efficiency. Figure 3 plots average MRR against token consumption per case. With one model call, 12.8K tokens, and 4.4 seconds per case, SIRCL⋆ is both more accurate and more efficient than all LLM-based baselines. OpenRCA, Flow-of-Action, LocaleXpert, and mABC require 6 to 49.2 calls per case and consume 17.5K to 515.9K tokens, yet all obtain lower average MRR. Even the most token-efficient LLM baseline, LocaleXpert, uses 6 calls and 17.5K tokens per case while reaching only 0.440 average MRR. The latency gap is also substantial: LLM-based baselines require 17.2 to 180.4 seconds per case, compared with 4.4 seconds for SIRCL⋆, reflecting the overhead of multi-step inference and intermediate analysis. Some classical methods are faster, but at substantially lower accuracy; for example, TORAI runs in 2.0 seconds per case, but its average MRR is 0.429, trailing SIRCL⋆ by 0.167.

C. RQ2: Selected Prompt Design

This subsection reports the development evaluation and validation results for the five prompt design variables defined in Section II. We discuss the selection decision for each variable in turn and conclude with a post-selection ablation study. In Tables V–IX, the ° marker denotes the SIRCL−0 default, bold marks the best development score $q^\star$ for selection, and the note below each table reports the validation decision.

TABLE VI: Modality-sequencing candidate evaluation.

	MRR by dataset			Qwen3.5-9B			DeepSeekv4-pro			Selection
Config	R-OB	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	Avg. MRR
Two modalities										
MET−TRC	0.965	0.388	0.265	0.524	0.457	0.596	0.555	0.501	0.607	0.539
TRC−MET	0.959	0.380	0.262	0.497	0.418	0.587	0.570	0.525	0.603	0.534
MET−LOG	0.954	0.389	0.370	0.531	0.457	0.616	0.612	0.554	0.668	0.571
LOG−MET	0.954	0.368	0.362	0.501	0.428	0.573	0.622	0.574	0.676	0.561
TRC−LOG	0.873	0.286	0.258	0.460	0.389	0.536	0.485	0.444	0.524	0.472
LOG−TRC	0.885	0.270	0.278	0.467	0.381	0.561	0.488	0.432	0.549	0.477
Three modalities										
MET−TRC−LOG°	0.971	0.401	0.340	0.545	0.474	0.626	0.596	0.541	0.648	0.571
MET−LOG−TRC	0.972	0.381	0.358	0.555	0.479	0.644	0.585	0.536	0.630	0.570
TRC−MET−LOG	0.959	0.405	0.311	0.544	0.470	0.623	0.573	0.520	0.624	0.559
TRC−LOG−MET	0.931	0.340	0.314	0.494	0.406	0.590	0.563	0.510	0.607	0.528
LOG−MET−TRC	0.964	0.402	0.366	0.560	0.474	0.658	0.595	0.547	0.639	0.577
LOG−TRC−MET	0.968	0.399	0.381	0.555	0.477	0.644	0.610	0.561	0.660	**0.583**

Decision: validate LOG−TRC−MET against the default MET−TRC−LOG; not significant (0.545→0.563, p=0.119), so retain MET−TRC−LOG.

Telemetry analysis. On the development set, the highest-scoring candidates are the metric z-score analyzer MET−Z, the exclusive span-latency analyzer TRC−L, and the keyword error-rate analyzer LOG−R (Table V). These analyzers replace the default choices MET−S, TRC−H, and LOG−T, respectively, and the resulting analyzer updates are all confirmed on the validation split (p<0.01).

The development results show two patterns. First, the best analyzer in each modality indicates that metric evidence is the strongest standalone signal (0.530 MRR), followed by traces (0.424) and logs (0.223), with the same ordering on both LLM backbones. Second, representation quality matters within each modality. The selected analyzers reduce telemetry to comparable per-candidate scores, whereas lower-scoring alternatives expose less aggregated distributions, dependency structures, or text counts that leave more aggregation to the model. This effect is largest for traces, where TRC−L substantially outperforms the raw latency-histogram representation TRC−H. The log analyzers are closer to one another, but LOG−R still provides the most compact cross-service signal among the tested log encodings. Overall, the analyzer stage suggests that SIRCL benefits from telemetry summaries that are already rankable before they are given to the LLM.

Modality sequencing. On the development set, the highest-scoring sequence is LOG−TRC−MET, with an average MRR of 0.583 (Table VI). The prespecified MET−TRC−LOG sequence reaches 0.571 MRR, matching the best two-modality sequence MET−LOG. However, on the validation split, the improvement from MET−TRC−LOG to LOG−TRC−MET is not significant (p=0.119). The sequential procedure therefore retains the default MET−TRC−LOG sequence.

The development results show that metric evidence is the most important modality for composition. All strong two-modality configurations include MET, while the metric-free pairs TRC−LOG and LOG−TRC are the weakest (0.472 and 0.477 MRR). The effect of trace evidence is dataset-dependent: compared with MET−LOG, MET−TRC−LOG improves R-OB

TABLE VII: Prompt-layout candidate evaluation.

	MRR by dataset			Qwen3.5-9B			DeepSeekv4-pro			Selection
Config	R-OB	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	Avg. MRR
U-BASE°	0.971	0.401	0.340	0.545	0.474	0.626	0.596	0.541	0.648	**0.571**
U-EARLY	0.970	0.399	0.338	0.552	0.482	0.634	0.586	0.533	0.634	0.569
S-BASE	0.963	0.406	0.317	0.536	0.480	0.593	0.588	0.536	0.643	0.562
S-TOPO	0.954	0.370	0.317	0.508	0.417	0.599	0.586	0.537	0.640	0.547
S-GROUP	0.975	0.370	0.254	0.477	0.404	0.553	0.589	0.543	0.629	0.533
S-GRAPH	0.979	0.394	0.280	0.507	0.453	0.558	0.595	0.556	0.624	0.551
S-RANK	0.937	0.339	0.275	0.470	0.398	0.543	0.565	0.512	0.614	0.517
S-MASK	0.820	0.345	0.226	0.459	0.368	0.567	0.468	0.384	0.546	0.464

Decision: U-BASE is the development-best candidate and is the SIRCL-0 default; no validation update is needed.

TABLE VIII: Auxiliary-guidance candidate evaluation.

	MRR by dataset			Qwen3.5-9B			DeepSeekv4-pro			Selection
Config	R-OB	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	Avg. MRR
NONE°	0.971	0.401	0.340	0.545	0.474	0.626	0.596	0.541	0.648	0.571
HEUR	0.976	0.434	0.325	0.551	0.497	0.609	0.606	0.562	0.649	**0.579**

Decision: validate HEUR against NONE; not significant (0.544→0.557, p=0.147), so retain NONE.

TABLE IX: Reasoning-instruction candidate evaluation.

	MRR by dataset			Qwen3.5-9B			DeepSeekv4-pro			Selection
Config	R-OB	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	Avg. MRR
NONE°	0.971	0.401	0.340	0.545	0.474	0.626	0.596	0.541	0.648	0.571
THINK	0.949	0.403	0.324	0.535	0.526	0.547	0.582	0.527	0.633	0.558
STEPS	0.967	0.406	0.346	0.551	0.467	0.630	0.595	0.518	0.666	0.573
CITE	0.950	0.460	0.361	0.574	0.513	0.645	0.607	0.534	0.685	0.590
VERIFY	0.974	0.485	0.384	0.588	0.531	0.669	0.641	0.584	0.717	**0.614**
WALK	0.945	0.404	0.322	0.528	0.438	0.642	0.585	0.495	0.691	0.557
TRIAGE	0.941	0.427	0.343	0.529	0.478	0.590	0.611	0.547	0.676	0.570
ROT	0.944	0.389	0.303	0.521	0.497	0.547	0.569	0.492	0.645	0.545

Decision: validate VERIFY against NONE; accept VERIFY (0.544→0.590, p<0.001).

TABLE X: SIRCL* ablation on test sets. Results are MRR.

Config.	**Aegis**	**R-TT**	**A-22**	**A-25**	**Macro avg.**	**Δ Avg.**
SIRCL*	0.635	0.993	0.379	0.378	0.596	–
a_{MET}: MET-Z → MET-S	0.588	0.961	0.274	0.345	0.542	−0.054
a_{TRC}: TRC-L → TRC-H	0.527	0.751	0.320	0.396	0.499	−0.097
a_{LOG}: LOG-R → LOG-T	0.550	0.994	0.358	0.325	0.557	−0.039
C_R: VERIFY → NONE	0.522	0.970	0.378	0.358	0.557	−0.039
SIRCL-0	0.292	0.692	0.173	0.161	0.330	−0.266

(0.954 to 0.971) and A-22 (0.389 to 0.401), but lowers A-25 (0.370 to 0.340). Ordering has a smaller effect than modality inclusion: the six three-modality sequences range from 0.528 to 0.583 MRR, and the highest-scoring order does not significantly improve over the default. Nevertheless, ordering is not negligible: the lowest-scoring three-modality sequence, TRC–LOG–MET, is significantly below LOG–TRC–MET on validation (p<0.001).

Prompt layout. As shown in Table VII, the default all-user layout U-BASE has the highest selection score on the development set (0.571 MRR), narrowly above U-EARLY (0.569). Since the default U-BASE is the top candidate, SIRCL* retains this configuration without requiring validation. The results suggest that layout has a smaller effect than telemetry representation or modality sequencing, although some reorganizations reduce accuracy. Moving background and output instructions to the system prompt does not help: S-BASE reaches 0.562 MRR, and S-TOPO drops to 0.547. Service-level grouping and ordering are more harmful for Qwen3.5-9B, reducing its MRR from 0.545 to 0.477 and 0.470, respectively. The graph-based layout S-GRAPH remains close to the default, whereas S-MASK has the largest drop, lowering average MRR from 0.571 to 0.464. This suggests that human-readable identifiers provide useful localization information in these benchmarks, rather than reflecting naming bias alone.

Auxiliary guidance. As shown in Table VIII, the literature-derived guidance HEUR slightly improves the development average over NONE (0.579 vs. 0.571 MRR). However, this improvement is not significant on the validation split (p=0.147). The sequential procedure therefore retains the default empty guidance block, C_{G} = NONE. Guidance mined from labeled prediction trajectories is evaluated separately in Section IV-E.

Reasoning instruction. The reasoning-instruction stage selects VERIFY (Table IX). On the development set, VERIFY obtains the highest selection score (0.614 MRR), improving over the direct-ranking default NONE (0.571). The validation comparison confirms the update from NONE to VERIFY, with MRR increasing from 0.544 to 0.590 (p<0.001). Thus, SIRCL* includes VERIFY as its reasoning instruction. Among the remaining candidates, CITE also improves over NONE, indicating that grounding candidates in evidence is beneficial. However, VERIFY performs better by explicitly checking and revising the top candidate before returning the final ranking.

Overall, the sequential procedure selects a compact configuration: SIRCL* replaces the default telemetry analyzers with MET-Z, TRC-L, and LOG-R, and adds the verification instruction VERIFY. The default modality sequence, prompt layout, and empty auxiliary-guidance block are retained because no alternative provides a validated improvement. These results suggest that, once telemetry representation is strong, more elaborate prompt organization or auxiliary guidance does not necessarily improve RCL. We next quantify how the selected changes contribute to the overall improvement.

Post-selection ablation. We evaluate the contribution of each selected change on the test set by starting from SIRCL* and reverting one decision at a time to its value in SIRCL-0 (Table X). Each reversion reduces the overall average MRR. The largest drop comes from reverting the trace analyzer from TRC-L to TRC-H (0.596→0.499), followed by the metric analyzer (0.596→0.542). Reverting the log analyzer and removing the verification instruction both reduce average MRR to 0.557. The full SIRCL-0 configuration drops further to 0.330, showing that the selected changes jointly account for a substantial improvement over the vanilla prompt. Although a few reversions slightly improve individual datasets, every selected change contributes positively to average performance.

TABLE XI: Comparison across seven LLM backbones on test sets. **Best** bold, second underlined.

	Aegis			R-TT			A-22			A-25			Macro average			Per-case cost		
LLM backbone	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	MRR	A@1	A@3	Time (s)	Tokens	$
Qwen3.5-9B	0.635	0.560	0.730	0.993	**0.989**	**1.000**	0.379	0.314	0.450	0.378	0.317	0.458	0.596	0.545	0.660	**4.4**	12.8k	**0.001**
DeepSeek-v4-pro	0.579	0.450	0.740	0.948	0.911	0.989	0.429	0.336	0.557	0.364	0.279	0.479	0.580	0.494	0.691	102.8	15.1k	0.014
Claude-Opus-4.7	**0.694**	**0.590**	**0.820**	**0.994**	**0.989**	**1.000**	0.514	0.443	0.593	0.416	0.329	**0.521**	**0.654**	**0.588**	0.733	15.2	14.9k	0.267
GPT-5.4	0.638	0.480	0.810	0.965	0.944	0.989	0.488	0.400	0.600	0.363	0.296	0.475	0.614	0.530	0.719	43.2	13.0k	0.300
GPT-5.4-mini	0.562	0.480	0.680	0.948	0.922	0.978	0.433	0.321	0.579	0.343	0.258	0.475	0.572	0.495	0.678	33.5	13.2k	0.036
Gemini-3.1-pro	0.566	0.470	0.690	0.974	0.956	**1.000**	**0.527**	**0.479**	0.600	**0.435**	**0.375**	**0.521**	0.626	0.570	0.703	56.1	20.7k	0.086
Kimi-K2.5	0.660	0.550	0.810	0.983	0.967	**1.000**	0.516	0.393	**0.664**	0.418	0.325	**0.521**	0.644	0.559	**0.749**	19.3	**10.4k**	0.007

D. RQ3: Transferability and Reproducibility

Backbone transferability. We evaluate the SIRCL* design on seven LLM backbones without further tuning (Table XI). The resulting macro-average MRR ranges from 0.572 to 0.654, indicating that the selected design remains effective across model families. Claude-Opus-4.7 achieves the best average MRR (0.654), followed by Kimi-K2.5 (0.644), Gemini-3.1-pro (0.626), and GPT-5.4 (0.614). Qwen3.5-9B reaches 0.596 average MRR, placing it within the performance range of the larger API models and ahead of DeepSeek-v4-pro (0.580) and GPT-5.4-mini (0.572). The ranking is not uniform across datasets: Gemini-3.1-pro performs best on A-22 and A-25, while R-TT is nearly saturated for all backbones. These results suggest that model choice affects final accuracy, but the selected prompt design remains effective across both smaller open models and larger proprietary models.

Efficiency differs more sharply across backbones. Qwen3.5-9B is served locally, so its measured latency reflects local deployment, while the latencies of the other backbones reflect provider API calls; for dollar cost, we estimate all models using provider-listed prices [40], [42]–[46], [48]. Under this accounting, Qwen3.5-9B is the most cost-efficient option: it has the lowest latency (4.4 s per case) and the lowest estimated cost ($0.001 per case), while consuming 12.8K tokens, second only to Kimi-K2.5 (10.4K). The larger API models provide higher peak accuracy but at substantially higher cost or latency: Claude-Opus-4.7 improves average MRR by 0.058 over Qwen3.5-9B, but takes 15.2 s and costs $0.267 per case. Differences in token consumption also reflect model-specific reasoning behavior under the same prompt, especially for models that generate longer internal reasoning traces. Overall, Qwen3.5-9B offers the best cost–latency trade-off, while Claude-Opus-4.7 and Kimi-K2.5 represent higher-accuracy alternatives when additional cost is acceptable.

Reproducibility. We assess reproducibility at both design selection and inference time. For design selection, we repeat the development–validation split procedure with ten random splits. The selected design is stable across all runs: SIRCL* is selected in 10 out of 10 splits, with the same analyzer choices, the same VERIFY instruction, and the same retained defaults. For inference stability, we use temperature 0 whenever the option is available to make the model outputs as deterministic as possible. We rerun the SIRCL* configuration with Qwen3.5-9B over ten passes on H100 GPUs. Across runs, the MRR standard deviation is 0.000, and the top-1 prediction is identical for every case.

TABLE XII: Evaluation of failure-mined auxiliary guidance candidates on A-22 and A-25 test sets. Best in bold.

	MRR by dataset		Qwen3.5-9B			DeepSeekv4-pro			Macro avg.
Config.	A-22	A-25	MRR	A@1	A@3	MRR	A@1	A@3	MRR
NONE	0.401	0.340	0.329	0.226	0.446	0.411	0.340	0.478	0.370
MINE	0.433	0.363	0.367	0.290	0.450	0.429	0.373	0.476	0.398
MINE-ALL	0.435	0.387	0.393	0.315	0.472	0.429	0.361	0.503	0.411
MINE-2X	0.447	0.347	0.355	0.274	0.436	0.439	0.345	0.520	0.397
MINE-G55	0.485	0.362	0.383	0.307	0.467	0.465	0.392	0.540	**0.424**

E. Additional Results

Failure analysis. We analyze RCL failures of SIRCL* on AegisLab, a hard benchmark in the test set, comparing Qwen3.5-9B with Claude-Opus-4.7. Claude reduces top-1 errors only modestly, from 44 to 41 cases, indicating that the remaining failures cannot be attributed to model scale alone. To characterize reasoning failures, each model produces a brief rationale with its RCL prediction, and we use GPT-5.4 to annotate the rationales for failed cases using the taxonomy of Riddell et al. [49]. Claude produces cleaner reasoning traces: logical-error prevalence drops from 0.89 to 0.46, and misattribution drops from 0.98 to 0.61. However, ungrounded claims remain pervasive for both backbones (1.00 vs. 0.98). Manual inspection suggests a recurring evidence-attribution bottleneck: when both the root-cause service and downstream symptom services show anomalies, the model often ranks the component with the larger observed anomaly higher, even when the call graph indicates that the anomaly is propagated.

Failure-mined guidance. Although mined auxiliary guidance is not eligible for C_G selection because the mining process uses RCL prediction trajectories labeled with ground truth from the target benchmark, it provides an auxiliary analysis of the additional gains possible when benchmark-specific offline mining is allowed. In this setting, all mined-guidance variants improve over the empty guidance NONE, increasing average MRR from 0.370 to 0.397–0.424 (Table XII). The strongest variant, MINE-G55, reaches 0.424 MRR; unlike the other mined variants, which use DeepSeek-v4-pro as the distiller, MINE-G55 uses GPT-5.5, suggesting that a stronger distillation model can produce more effective diagnostic rules. Mining from both correct and incorrect trajectories also helps:

MINE-ALL reaches 0.411 MRR, higher than the default MINE at 0.398. By contrast, simply increasing the number of failure samples is less reliable, as MINE-2X reaches only 0.397 MRR. These results suggest that an offline failure-analysis process can provide useful diagnostic guidance, but we exclude it from SIRCL* to preserve the single-call setting without benchmark-specific labeled trajectories.

V. Threats to Validity

External threats. Our evaluation uses five public microservice benchmarks constructed from controlled fault-injection experiments, including synthetic and semi-synthetic faults. These benchmarks provide repeatable telemetry and ground-truth labels, but may not capture the full diversity of production incidents or telemetry noise. Adapting SIRCL to production deployments may require incident-type-specific prompt variants, additional telemetry context, or calibration to local observability conventions, which are beyond the scope of this work. We report A@1 and A@3 alongside MRR and average across benchmark cases, rather than weighting incidents by their real-world operational frequency. Reported wall-clock latencies also depend on serving conditions, especially for hosted model backbones.

Internal threats. The sequential selection procedure fixes one design variable at a time, so SIRCL* is not guaranteed to be the best configuration in the full Cartesian-product search space. Instead, it is the result of a budgeted, validation-driven search over a finite and curated candidate set. The selected configuration may also depend on the initial default design and on the candidate choices included for analyzers, layouts, guidance, and reasoning instructions. To reduce backbone-specific selection effects, we select using one smaller open model and one larger proprietary model under a fixed budget. The nested protocol in Section IV-A further limits selection bias: each decision is chosen on development incidents, validated on a held-out subset over ten random splits, and evaluated only after locking the final design on withheld test sets. In the baseline comparison, using a shared Qwen3.5-9B backbone helps isolate workflow differences from model differences, but it does not imply that the relative ranking of agentic baselines is invariant across all possible LLMs. We therefore separately evaluate the locked SIRCL* design across multiple backbones to assess model sensitivity (Section IV-D).

VI. Related Work

Non-LLM based RCL. Classical microservice RCL performs causal inference or graph-based analysis using one or more telemetry modalities [1], [4], [5]. Metrics-based methods build causal or dependency graphs over service metrics and rank candidate root causes through intervention recognition [22], [34], causal structure discovery [50], [51], or graph traversal [26], [51], [52]. Trace-based methods analyze request paths and conduct spectrum analysis on the services that appear in anomalous and normal traces [25], [53], [54] to rank suspicious services. Log-based methods typically parse unstructured logs into structured templates and detect abnormal log-event patterns [27], [55], [56] or construct causal graphs [57]. Multi-modal methods integrate traces, logs, metrics, and other signals to infer root causes from cross-modal evidence. They typically model cross-modal dependencies through graph-based propagation analysis [23], [37], learned telemetry representations [38], [58], or statistical and causal ranking over anomaly patterns [21], [36]. In contrast, SIRCL serializes the heterogeneous telemetry into one prompt and delegates the cross-modal reasoning to a single LLM inference.

LLM-based RCL. Early studies show that frozen LLMs can diagnose incidents from a single prompt [14], [15], and subsequent systems enrich that prompt with additional context or supervision. *RCACopilot* [16] relies on a database of historical incidents, *COCA* [59] incorporates code information, and *LasRCA* [60] trains a classifier for preliminary analysis using labeled data. More recent work spends additional computation per incident, either at inference time through multi-turn agentic workflows or ahead of time through fine-tuning. Agentic systems treat RCL as sequential tool use over telemetry: *OpenRCA* [11] writes and executes code across many turns, while *Flow-of-Action* [9], *mABC* [10], *RCAgent* [61], and *LocaleXpert* [28] assign predefined agent roles that orchestrate a multi-turn analysis loop, at the cost of many calls and a large token budget per incident. Fine-tuned systems instead pay a one-time training cost for accuracy: *FoundRoot* [13] learns structured reasoning through warm-up supervised fine-tuning and dynamic sampling policy optimization. *ThinkFL* [20] combines fine-tuning with a multi-turn tool-calling loop, but targets individual trace anomalies rather than the system-level incidents considered in SIRCL. In contrast, SIRCL keeps the frozen-model, single-inference setting and focuses on systematically selecting the prompt design itself.

Systematic prompt design. Prompting has been empirically shown to have a large effect on LLM performance [62], [63], and the prompt design space is correspondingly broad [64]: prior work studies instruction placement and long-context effects [65], component ordering [63], [66], [67], instruction optimization [68], [69], and evidence context engineering [19], [70], [71]. In software engineering, existing studies either refine prompt instructions [72], [73] or search over facts to include in the prompt [74]. For RCL, eARCO [17] optimizes prompt instructions and retrieves in-context examples from historical incidents, but leaves the telemetry-evidence representation and prompt organization fixed. SIRCL instead studies prompt design for RCL as a joint choice over the instruction and the evidence context, enabling a single-inference prompt selected without historical-incident retrieval or model fine-tuning.

VII. Conclusions

We present SIRCL, a systematic prompt-design framework that studies microservice root cause localization as a single-inference problem over a frozen language model. SIRCL casts prompt construction as a search over five decisions spanning telemetry evidence, prompt organization, auxiliary guidance, and reasoning instructions, and fixes these decisions through

a sequential development–validation protocol. The selected SIRCL* design enables a frozen open-weight 9 B model to reach 0.596 MRR across four held-out test sets with a single model call, achieving the best macro-average performance while using substantially fewer tokens than agentic LLM baselines. The same design transfers across seven LLM backbones.

The per-decision analysis shows where the gains come from: rankable telemetry representations matter more than raw evidence volume, the default three-modality sequence remains competitive after validation, and a lightweight verification instruction improves the final ranking. Further failure analysis suggests that remaining errors often involve evidence attribution, especially when propagated symptoms exhibit stronger anomalies than the true origin. Future work will explore how to address these cases through tool calls, follow-up prompts, or fine-tuning when single-inference prompting is insufficient.

REFERENCES

[1] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, and D. Pei, "Failure diagnosis in microservice systems: A comprehensive survey and analysis," *ACM Transactions on Software Engineering and Methodology*, 2025.

[2] N. Fu, G. Cheng, Y. Teng, G. Dai, S. Yu, and Z. Chen, "Intelligent root cause localization in microservice systems: A survey and new perspectives," *ACM Computing Surveys*, 2025.

[3] A. Fang, S. Zhang, Y. Yang, H. Wu, J. Xu, X. Wang, R. Wang, M. Wang, Q. Lu, and P. He, "Rethinking the evaluation of microservice rca with a fault propagation-aware benchmark," *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, 2026.

[4] A. Fang, H. Yang, H. Dong, Q. Lu, J. Xu, and P. He, "A goal-driven survey on root cause analysis," *arXiv preprint arXiv:2510.19593*, 2025.

[5] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P. Yu, and Y. Li, "A survey of aiops in the era of large language models," *ACM Computing Surveys*, 2025.

[6] L. Pham, H. Ha, and H. Zhang, "Root cause analysis for microservice system based on causal inference: How far are we?" in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024.

[7] ——, "Baro: Robust root cause analysis for microservices via multivariate bayesian online change point detection," *Proceedings of the ACM on Software Engineering*, 2024.

[8] L. Zhang, T. Jia, Y. Zhai, L. Pan, C. Duan, M. He, M. Jia, and Y. Li, "Agentic memory enhanced recursive reasoning for root cause localization in microservices," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice*, 2026.

[9] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li *et al.*, "Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis," in *Companion Proceedings of the ACM on Web Conference 2025*, 2025.

[10] W. Zhang, H. Guo, J. Yang, Z. Tian, Y. Zhang, Y. Chaoran, Z. Li, T. Li, X. Shi, L. Zheng, and B. Zhang, "mABC: Multi-agent blockchain-inspired collaboration for root cause analysis in micro-services architecture," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.

[11] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "Openrca: Can large language models locate the root cause of software failures?" in *The thirteenth international conference on learning representations*, 2025.

[12] Y. Tian, Y. Liu, Z. Chong, Z. Huang, and H.-A. Jacobsen, "Gala: Can graph-augmented large language model agentic workflows elevate root cause analysis?" *arXiv preprint arXiv:2508.12472*, 2025.

[13] Z. Xie, Z. Li, X. He, S. Zhang, L. Xu, Y. Yang, T. Zhang, J. Chen, R. Shi, and D. Pei, "Foundroot: Towards foundation model for root cause analysis via structured deep thinking," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.

[14] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering*, 2023.

[15] X. Zhang, S. Ghosh, C. Bansal, R. Wang, M. Ma, Y. Kang, and S. Rajmohan, "Automated root causing of cloud incidents using in-context learning with gpt-4," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024.

[16] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024.

[17] D. Goel, R. Magazine, S. Ghosh, A. Nambi, P. Deshpande, X. Zhang, C. Bansal, and S. Rajmohan, "earco: Efficient automated root cause analysis with prompt optimization," *arXiv preprint arXiv:2504.11505*, 2025.

[18] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, 2022.

[19] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang *et al.*, "A survey of context engineering for large language models," 2025.

[20] L. Zhang, Y. Zhai, T. Jia, C. Duan, S. Yu, J. Gao, B. Ding, Z. Wu, and Y. Li, "Thinkfl: Self-refining failure localization for microservice systems via reinforcement fine-tuning," *ACM Transactions on Software Engineering and Methodology*, 2025.

[21] L. Pham, H. Ha, X. Zhang, and H. Zhang, "Torai: Multi-source root cause analysis for blind spots in microservice service call graph," *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, 2026.

[22] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," *Advances in Neural Information Processing Systems*, 2022.

[23] L. Tao, X. Lu, S. Zhang, J. Luan, Y. Li, M. Li, Z. Li, Q. Yu, H. Xie, R. Xu *et al.*, "Diagnosing performance issues for large-scale microservice systems with heterogeneous graph," *IEEE Transactions on Services Computing*, 2024.

[24] R. Ding, C. Zhang, L. Wang, Y. Xu, M. Ma, X. Wu, M. Zhang, Q. Chen, X. Gao, X. Gao *et al.*, "Tracediag: Adaptive, interpretable, and efficient root cause analysis on large-scale microservice systems," in *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*, 2023.

[25] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang *et al.*, "Practical root cause localization for microservice systems via trace analysis," in *2021 IEEE/ACM 29th International Symposium on Quality of Service*, 2021.

[26] C. Hou, T. Jia, Y. Wu, Y. Li, and J. Han, "Diagnosing performance issues in microservices with heterogeneous data source," in *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking*, 2021.

[27] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *2017 IEEE international conference on web services*, 2017.

[28] Z. Zhong, R. Fu, M. Ma, S. Zhang, Y. Sun, C. Bansal, and D. Pei, "Llm-enhanced failure localization in microservices: Integrating multi-modal data and expert interpretation," *IEEE Transactions on Services Computing*, 2026.

[29] OpenAI. (2026) Gpt-5.5 model card. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-5.5

[30] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of the association for computational linguistics: ACL 2024*, 2024.

[31] Zenodo, "Aiops 2022 companion dataset," https://zenodo.org/records/19176851, 2026.

[32] A. Benchmark, "Aiopschallenge 2025 dataset," https://www.aiops.cn/gitlab/aiops-live-benchmark/aiopschallenge2025, 2025.

[33] S. Taheri and G. Hesamian, "A generalization of the wilcoxon signed-rank test and its applications," *Statistical Papers*, 2013.

[34] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, "Causal inference-based root cause analysis for online service systems with

intervention recognition," in *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*, 2022.

[35] Y. Meng, S. Zhang, Y. Sun, R. Zhang, Z. Hu, Y. Zhang, C. Jia, Z. Wang, and D. Pei, "Localizing failure root causes in a microservice through causality inference," in *2020 IEEE/ACM 28th International Symposium on Quality of Service*. IEEE, 2020.

[36] Z. Zhu, C. Lee, X. Tang, and P. He, "Hemirca: Fine-grained root cause analysis for microservices with heterogeneous data sources," *ACM Transactions on Software Engineering and Methodology*, 2024.

[37] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering*, 2023.

[38] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *2023 IEEE/ACM 45th International Conference on Software Engineering*, 2023.

[39] S. Zhang, P. Jin, Z. Lin, Y. Sun, B. Zhang, S. Xia, Z. Li, Z. Zhong, M. Ma, W. Jin *et al.*, "Robust failure diagnosis of microservice system through multimodal data," *IEEE Transactions on Services Computing*, 2023.

[40] DeepSeek. (2026) Deepseek-v4-pro model card. [Online]. Available: https://api-docs.deepseek.com/quick_start/pricing

[41] Qwen. (2026) Qwen-3.5-9b model card. [Online]. Available: https://huggingface.co/Qwen/Qwen3.5-9B

[42] MoonshotAI. (2026) Kimi-k2.5 model card. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-moonshot-ai-kimi-k2-5.html

[43] OpenAI. (2026) Gpt-5.4 model card. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-5.4

[44] ——. (2026) Gpt-5.4-mini model card. [Online]. Available: https://developers.openai.com/api/docs/models/gpt-5.4-mini

[45] Google. (2026) Gemini-3.1-pro model card. [Online]. Available: https://ai.google.dev/gemini-api/docs/pricing

[46] Anthropic. (2026) Claude-opus-4-7 model card. [Online]. Available: https://www.anthropic.com/news/claude-opus-4-7

[47] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[48] OpenRouter. (2026) Qwen-3.5-9b api price. [Online]. Available: https://openrouter.ai/qwen/qwen3.5-9b

[49] E. Riddell, J. Riddell, G. Sun, M. Antkiewicz, and K. Czarnecki, "Stalled, biased, and confused: Uncovering reasoning failures in llms for cloud-based root cause analysis," in *Proceedings of the 3rd International Conference on AI Foundation Models and Software Engineering*, 2026.

[50] R. Xin, P. Chen, and Z. Zhao, "Causalrca: Causal inference based precise fine-grained root cause localization for microservice applications," *Journal of Systems and Software*, 2023.

[51] C.-M. Lin, C. Chang, W.-Y. Wang, K.-D. Wang, and W.-C. Peng, "Root cause analysis in microservice using neural granger causal discovery," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[52] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "Microrca: Root cause localization of performance issues in microservices," in *IEEE/IFIP Network Operations and Management Symposium*, 2020.

[53] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, and X. Li, "Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments," in *Proceedings of the Web Conference 2021*, 2021.

[54] L. Tao, X. Lu, S. Zhang, J. Luan, Y. Li, M. Li, Z. Li, Q. Yu, H. Xie, R. Xu *et al.*, "Diagnosing performance issues for large-scale microservice systems with heterogeneous graph," *IEEE Transactions on Services Computing*, 2024.

[55] P. Notaro, S. Haeri, J. Cardoso, and M. Gerndt, "Logrule: Efficient structured log mining for root cause analysis," *IEEE Transactions on Network and Service Management*, 2023.

[56] S. He, Q. Lin, J.-G. Lou, H. Zhang, M. R. Lyu, and D. Zhang, "Identifying impactful service system problems via log analysis," in *Proceedings of the 2018 26th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2018.

[57] M. Markakis, B. Youngmann, T. Gao, Z. Zhang, R. Shahout, P. B. Chen, C. Liu, I. Sabek, and M. Cafarella, "From logs to causal inference: diagnosing large systems," *Proceedings of the VLDB Endowment*, 2024.

[58] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Zhang, "Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proceedings of the 44th international conference on software engineering*, 2022.

[59] Y. Li, Y. Wu, J. Liu, Z. Jiang, Z. Chen, G. Yu, and M. R. Lyu, "Coca: Generative root cause analysis for distributed systems with code knowledge," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025.

[60] Y. Han, Q. Du, Y. Huang, J. Wu, F. Tian, and C. He, "The potential of one-shot failure root cause analysis: Collaboration of the large language model and small classifier," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024.

[61] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "Rcagent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," 2024.

[62] A. Salinas and F. Morstatter, "The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[63] Y. Mao, J. He, and C. Chen, "From prompts to templates: A systematic prompt template analysis for real-world llmapps," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025.

[64] X. Zhang, J. Cao, C. You, and D. Ding, "Why prompt design matters and works: A complexity analysis of prompt search space in llms," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.

[65] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, 2024.

[66] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in *Extended abstracts of the 2021 CHI conference on human factors in computing systems*, 2021.

[67] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff *et al.*, "The prompt report: A systematic survey of prompt engineering techniques," *arXiv preprint arXiv:2406.06608*, 2024.

[68] T. Shin, Y. Razeghi, R. L. L. Iv, E. Wallace, and S. Singh, "Autoprompt: Eliciting knowledge from language models with automatically generated prompts," in *Proceedings of the 2020 conference on empirical methods in natural language processing*, 2020.

[69] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang *et al.*, "A systematic survey of automatic prompt optimization techniques," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.

[70] F. Cuconasu, G. Trappolini, F. Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonellotto, and F. Silvestri, "The power of noise: Redefining retrieval for rag systems," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

[71] F. Sovrano, G. Dominici, and A. Bacchelli, "Mitigating prompt-induced cognitive biases in general-purpose ai for software engineering," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.

[72] M. Tafreshipour, A. Imani, E. Huang, E. S. de Almeida, T. Zimmermann, and I. Ahmed, "Prompting in the wild: An empirical study of prompt evolution in software repositories," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories*, 2025.

[73] E. Santana Jr, G. Benjamin, M. Araujo, H. Santos, D. Freitas, E. Almeida, P. A. d. Neto, J. Li, J. Chun, and I. Ahmed, "Which prompting technique should i use? an empirical investigation of prompting techniques for software engineering tasks," *arXiv preprint arXiv:2506.05614*, 2025.

[74] N. Parasaram, H. Yan, B. Yang, Z. Flahy, A. Qudsi, D. Ziaber, E. T. Barr, and S. Mechtaev, "The fact selection problem in llm-based program repair," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025.